ChatGPT

Adding HCL Support to Go-Task (Taskfile) and a Basic HCL LSP

Background: YAML Pain Points in Taskfile

Go-Task's **Taskfile** is currently written in YAML and uses Go text templating for dynamic behavior. This approach has grown unwieldy – users have noted that Taskfile YAML is effectively becoming "YAML as a Programming Language" with heavy reliance on string templates [1]. Complex quoting, escaping, and the limited error handling of text templates make the Taskfile hard to write and debug [1]. YAML is meant for data serialization, not for rich logic, so trying to implement functions or conditionals leads to convoluted syntax. For example, referencing one variable in another requires inscrutable `{{.VARNAME}}` template expressions, and calling utility functions (via Sprig) embeds logic into strings. These pain points motivate moving to **HashiCorp Configuration Language (HCL)**, which was **designed for configuration** and includes first-class support for interpolation, expressions, and functions.

HCL (used by Terraform) strikes a balance between data and code: "Whereas JSON and YAML are formats for serializing data structures, HCL is a syntax and API specifically designed for building structured configuration formats... designed to be easily read and written by humans, and allows declarative logic for more complex applications" [2]. In other words, HCL allows embedding logic (expressions, conditionals, etc.) directly in config files without resorting to ad-hoc templating. HCL's native syntax includes an expression language so that **variables and functions can be used for dynamic configuration** [3]. This means in an HCL Taskfile we can use variables and call helper functions *natively* (e.g. string manipulation or shell commands) instead of awkward string substitutions. Docker's Buildx "bake" feature is a great example: it supports YAML or JSON, but **recommends HCL because it supports a more feature-complete spec** (native variables, expressions, etc.) [4]. In fact, **HCL is the preferred format for Bake files** since it unlocks features that JSON/YAML can't [4]. We intend to bring these same benefits to Taskfile by adding HCL support, while **keeping the existing Task engine intact**.

Goal: Add HCL as a First-Class Taskfile Format

We will extend the Taskfile system to accept HCL syntax (alongside YAML) for defining tasks. The core **execution engine and task model will remain the same** – we are only introducing a new input format. The plan is to parse `.hcl` Taskfiles and convert them into the same internal structs that YAML Taskfiles use. This way, all the task execution logic, dependency resolution, etc., can be reused unchanged (the Go-Task engine is well-tested and "has a really good setup" so we avoid touching it). YAML support will remain fully intact for backward compatibility. The **HCL and YAML formats should be interoperable**: it should be possible to include a YAML Taskfile from an HCL one (and vice versa) seamlessly. In other words, teams can gradually adopt HCL without rewriting everything at once – a parent Taskfile in HCL could `include` a legacy YAML Taskfile, or a YAML Taskfile could include a new HCL snippet. The *ultimate goal* is that **both formats feed into the *exact same code path*** internally, ensuring consistent behavior.

Why HCL? HashiCorp's HCL is a proven configuration language from the Terraform ecosystem that will give Taskfile a more natural, programming-like syntax *without* needing a full programming language. HCL is specifically built for config, not general computation, which makes it a good fit here. As the HCL README notes, it "allows declarative logic to permit its use in more complex applications" [2], while still being human-friendly. It has a native expression engine for arithmetic, string interpolation, conditionals, and can be extended with custom functions [5]. We can leverage those capabilities in Taskfile to replace the "insane string templating" currently used. For example, in HCL we could write:

```
variable "RECIPIENT" {
  default = "World"
}

task "greet" {
  cmds = ["echo \"Hello, ${RECIPIENT}!\""]
}
```

This defines a variable with a default and uses it in a task command via `${RECIPIENT}` interpolation. In the current YAML format, the equivalent is:

```
vars:
  RECIPIENT: '{{ default "World" .RECIPIENT }}'
tasks:
  greet:
    cmds:
      - echo "Hello, {{.RECIPIENT}}!"
```

The HCL version is clearly easier to read and avoids the double-curly template syntax. **Native interpolation** means the HCL parser and runtime handle replacing `${RECIPIENT}` with its value, and we can even call functions (e.g. `${upper(RECIPIENT)}` to uppercase, if we expose an `upper()` function). This demonstrates how HCL will **use native templating and function calls** instead of ad-hoc text templating.

HCL Taskfile Syntax Design

We will design an HCL structure for Taskfiles that closely mirrors the YAML schema (so that we map to the same internal structs). The top-level of an HCL Taskfile can define global settings and then tasks as labeled blocks. A likely approach:

- **Version and Global Settings:** At top level, we'll allow attributes like `version = "3"` (Taskfile version), `output = "group"` (output style), `method = "checksum"` (default method), etc., just as in YAML. These can be simple HCL attributes since they're singular values [6]. Global `vars` and `env` can also be defined at top-level (perhaps as blocks or maps – see below). We may also allow a top-level `dotenv` list attribute and other global keys from the YAML schema [7].

- **Task Definitions:** Each task will be an HCL **block**. For example:

```
task "build" {
  desc    = "Compile the project"
  deps    = ["init"]           # run "init" task first
  vars    = { GOOS = "linux" }  # task-specific vars
  cmds    = ["go build -o bin/app ./..."]
}
```

Here `task "build" { ... }` defines a task named **build**. Inside the block, the fields correspond to the Task struct: description, dependencies, vars, env, commands, etc. We'll support all task attributes that YAML supports (e.g. `cmds`, `deps`, `preconditions`, `dir`, `env`, `silent`, `method`, `ignore_error`, etc. – these map directly to our internal `Task` struct [8] [9]). Simple attributes like `desc` or `dir` become HCL attributes within the block. List attributes (like `cmds`, `deps`, `sources`, etc.) will be HCL lists. For example, `deps = ["task1", "task2"]` or multi-line list syntax. If a dependency needs to carry vars or be silent, we can use an object or nested block form (see **Dependency Handling** below). Overall, the **hierarchy** is similar to Terraform or Docker Bake: tasks are blocks identified by name. This is more natural in HCL than a YAML map of maps.

- **Variables Definition:** We need to decide how to define global and task-level variables in HCL. HCL offers a few options:
- We could use a **top-level** `variable` **block** (like Terraform) for each global variable. For example, `variable "FOO" { default = "value" }` would set a global var. Docker Bake uses this style for defining build variables [10]. This is clear and gives each variable its own block (and we can later extend with type or validation if needed). Alternatively, we could allow a shorthand map of variables (e.g. a top-level `vars` attribute with a map). However, HCL tends to encourage separate blocks for better structure and documentation. For **task-scoped** variables (the `vars:` under a task in YAML [11]), we can similarly allow a `vars` attribute containing a map, or consider supporting nested `var` blocks. The simplest approach is to treat `vars` as an attribute whose value is an HCL object/map of key-value pairs. Static values in that map are straightforward (strings, numbers, bools, lists, etc. get set as static vars). We will need a convention for dynamic (shell) variables – see **Dynamic Variables** below.

- **Environment variables** (`env:` in YAML) can be handled similarly to vars. We'll have a top-level `env` map (global env) and allow an `env` attribute in each task block for task-specific environment vars [11]. These would decode to the same internal `Vars` structure.

- **Includes:** YAML Taskfiles support an `includes` map to incorporate other Taskfiles [12]. We'll enable the same concept in HCL, likely with a block-based syntax. For instance, we could introduce an `include` **block** or use HCL's map-of-object ability. An HCL example might be:

```
include "common" {
  taskfile = "path/to/Taskfile.yml"
  dir      = "../"        # optional working dir for included file
  optional = false        # other include flags...
```

```
  vars = { ENV = "prod" } # variables to pass to included Taskfile
}
```

Each `include "<alias>" { ... }` block would correspond to an entry in the YAML `includes:` map (with alias as key) [13]. We will parse these and merge included tasks exactly as Task does today (Task's include mechanism will remain the same under the hood [12], we're just providing an HCL way to specify it). Crucially, **includes should accept either YAML or HCL** files: the code will detect the included file's extension and parse accordingly (e.g., `.yaml/.yml` goes through YAML loader, `.hcl` goes through HCL loader). This ensures **cross-compatibility** – a YAML Taskfile can include an HCL Taskfile by simply pointing to it (and vice versa). We will enforce that included HCL files declare no further includes themselves (the same rule as YAML includes to avoid recursion) [14].

Dependency and Command Handling: In YAML, tasks can depend on others either by name or with additional parameters (each dependency can have `task: name`, `vars: {…}`, `silent: true` etc.) [15]. Similarly, each command in `cmds` can either be a simple string or an object (if it calls another task or uses `defer` or `for` loops) [16] [17]. We will design the HCL schema to accommodate these cases: - For **dependencies**, we have two options. Easiest is to allow `deps` to be a list of strings for simple deps, *or* a list of dependency objects for cases with vars/silent. In HCL, a list can contain objects (maps) with keys matching our `Dependency` struct (e.g. `{ task = "build", vars = {…}, silent = true }`). The HashiCorp `gohcl` decoder can likely handle a `deps []Dependency` where Dependency has fields `Task`, `Vars`, `Silent` [15], and accept either a plain string or object. If needed, we can implement a custom decode to support strings as a shorthand (e.g. string `"foo"` gets mapped to `task="foo"` internally). Alternatively, we could introduce **nested blocks** for dependencies (like `dep "name" { vars = {...} }`), but this is less idiomatic in HCL (since dependencies are not a free-form resource, just a field of task). We will likely stick to `deps = ["dep1", { task = "dep2", vars = {...} }]` style for simplicity. The same principle applies to **commands**: a `cmds` list can contain plain strings (shell commands) or objects for special commands. For example, calling another task could be `cmds = [{ task = "other-task", vars = {Foo="bar"} }]` which corresponds to YAML's `- task: other-task` entry [18]. We will ensure our HCL decoding logic can interpret these correctly into the internal `Cmd` structs (which have fields for `Cmd`, `Task`, `Vars`, `defer`, etc. [18]).

Example: To illustrate, here's a small example comparing YAML vs HCL:

- *YAML Taskfile:*

```
version: "3"
vars:
  NAME: "Alice"
tasks:
  greet:
    deps: [setup]              # simple dependency
    cmds:
      - echo "Hello, {{.NAME}}!"
  setup:
```

```
    cmds:
      - echo "Setting up..."
```

- *HCL Taskfile (proposed):*

```
version = "3"
variable "NAME" {
  default = "Alice"
}

task "greet" {
  deps = ["setup"]
  cmds = ["echo \"Hello, ${NAME}!\""]
}

task "setup" {
  cmds = ["echo \"Setting up...\""]
}
```

Both achieve the same result, but the HCL uses `${NAME}` interpolation instead of `{{.NAME}}`. The HCL is more concise (no quotes escaping within `{{ }}`) and leverages the config language's features.

Implementation Plan Overview

To implement HCL support, we will create a **fork of the go-task repository** and make the following additions/modifications:

1. Recognize `.hcl` Taskfile Format

We will update Task's file-finding logic to detect HCL files. Currently Task looks for `Taskfile.yml`, `Taskfile.yaml`, etc. [19]. We will extend this to include `Taskfile.hcl` (and maybe `taskfile.hcl` lowercase, plus `.dist.hcl` variants for consistency). The search order can place `.hcl` either at the top or after YAML files – likely after YAML for backward compatibility (so existing repos aren't broken by accidentally having an HCL file around). The user can also explicitly specify a Taskfile with `-t file.hcl`. Once identified, we'll load the HCL file with a new parser.

2. Parse HCL into Internal Structures

We will use HashiCorp's **HCL library (v2)** to parse and decode the Taskfile. HashiCorp provides high-level Go APIs to decode HCL into Go structs, e.g. the `hclsimple.DecodeFile` or `gohcl` package [20] [21]. This will greatly simplify our work: we define Go structs mirroring the Taskfile schema and let the library populate them from the HCL file. Conveniently, our internal `Taskfile`, `Task`, `Var`, etc., types already

exist (in `internal/taskfile` package). We can leverage them or define parallel ones for decoding if needed. The approach:

- **Add HCL Struct Tags:** To use `hclsimple` or `gohcl`, we annotate our structs with `hcl` tags to describe the HCL schema. For example, in the `Taskfile` struct we might add tags like:

```
type Taskfile struct {
    Version    string            `yaml:"version" hcl:"version"`
    Output     string            `yaml:"output,omitempty"
hcl:"output,optional"`
    Method     string            `yaml:"method,omitempty"
hcl:"method,optional"`
    Includes   map[string]Include `yaml:"includes,omitempty"
hcl:"include,block"`
    Vars       Vars              `yaml:"vars,omitempty" hcl:"vars,remain"`
    Env        Vars              `yaml:"env,omitempty" hcl:"env,remain"`
    Tasks      Tasks             `yaml:"tasks" hcl:"task,block"`
}
```

  This is illustrative – the exact tags will depend on how we decide to structure the blocks. We see that `gohcl` allows us to mark block types with a label (e.g. `task,block` means the `Tasks` map is filled by `task "<name>" { }` blocks) [22] [23]. HashiCorp's own example shows how a struct can capture labeled blocks and attributes [22]. We will do similarly:
- Define that each `task` block in HCL (with a name label) goes into the `Tasks` map keyed by name. HashiCorp's decoder can use a map or a slice with label fields; using a `map[string]*Task` (Tasks) with `hcl:"task,block"` is feasible if we implement a custom decode or gather by name. If that's complex, we might decode into a slice of an intermediate struct that has a `Name` field and a `Task` inside, then map it.
- For `Includes`, we likely need a slice or map of `Include` blocks. Possibly `Includes map[string]Include` with `hcl:"include,block"` and inside the block use the label as the map key. We can use the `label` tag on a field inside `Include` struct for the alias name.
- The `Vars` and `Env` fields (maps of variables) are trickier because we want to allow both **inline values and objects**. We might mark them with `,remain` to accept arbitrary content and then post-process. Another strategy is to use a **custom decoder**: our `Var` struct could implement the HCL decoding interface to accept either a string (for static) or an object with `sh` field (for dynamic) – similar to how we implemented `UnmarshalYAML` in the YAML version. If this is too much upfront, we can initially support only static values in HCL (requiring dynamic shell usage to go through a function call as discussed later). In any case, the HCL library allows decoding into interface{} or raw messages if needed, which we can then interpret.

Using **hclsimple** is straightforward: we can call `hclsimple.DecodeFile("Taskfile.hcl", nil, &taskfileStruct)` to parse and populate our struct (the library handles lexing, parsing, and decoding in one call) [23] [24]. It will throw an error if the HCL is syntactically invalid or doesn't match the expected schema, which we can then report to the user (including line numbers). The **key advantage** is we don't have to write a full parser – we just declare the schema via struct tags, and HCL's library will verify the config and fill the data. This aligns with how HCL is meant to be used: *"The application defines which attribute names*

and block types are expected, and HCL parses the file, verifies it conforms to that structure, and returns high-level objects for the application" [25]. We will be effectively treating our existing Taskfile struct definitions as the schema.

If adding `hcl:` tags directly to the existing structs is intrusive, we can alternatively define parallel struct types for decoding (e.g. an `HclTaskfile` that mirrors Taskfile fields with HCL tags) and then convert that to the real Taskfile struct. However, adding tags is likely fine and keeps maintenance simpler (ensuring any new Taskfile fields in the future support both YAML and HCL).

3. Minimal Changes Outside Parsing

Our aim is **to avoid altering the core execution code**. After parsing, we will have a populated `Taskfile` struct (the same struct that YAML parsing produces). We can then pass it to the same execution engine (runner) as usual. All task dependencies, variables, commands will be executed exactly as if they came from YAML. This means if our HCL parser properly sets the `Vars`, `Cmds`, etc., the rest of Task doesn't need to know whether the source was YAML or HCL. For example, the logic that expands variables in commands can remain unchanged. Task's runtime already handles static vs dynamic vars (the `Var` type has either a Static string or a shell command to run in `Sh` field) and template expansion for those. We will reuse that mechanism: HCL-introduced variables that are static will be in `Var.Static`, and dynamic ones (if implemented via `Sh`) will be in `Var.Sh`, so that when a task runs, if `Var.Sh` is set, Task will execute that shell to get the value [26] [27]. In other words, **we want HCL ingestion to populate the same internal structures and flags that YAML ingestion does**, to keep behavior consistent.

The only non-parsing logic we might adjust is the **include resolution**: when merging included Taskfiles, currently Task likely calls the YAML loader recursively. We will modify that to call the appropriate loader based on file extension. For instance, if `includes: { foo: "subtasks.hcl" }`, our loader should detect `.hcl` and call the HCL decode for that file. The merging (`taskfile.Merge()` logic) will remain the same [28] – it merges two Taskfile structs (applying namespace, flattening, etc.). As such, **if too many non-YAML code paths needed changes**, it might not be worth it (as the user alluded), but we expect we can integrate HCL with minimal impact: mostly confined to the input reading stage and small conditional branches for format.

4. Variable and Function Resolution Timing

One crucial aspect is ensuring **HCL variables and functions resolve at the right time – i.e., when the task is executed, not prematurely**. In YAML Taskfiles, if you define `vars: FOO: {sh: "command"}`, Task does *not* run that command at parse time; it defers it until the variable is needed (each time the task runs) [26]. We need similar lazy evaluation in HCL for dynamic values. HCL by default will try to evaluate all expressions during parsing (since it's meant to produce a final config). For example, if you had `count = exec("somecommand")` in Terraform, it would execute at plan time. We want to avoid executing user shell commands just by reading the Taskfile – they should run only in context of their task.

To achieve this, we can leverage HCL's expression system and custom functions. Specifically, we will **introduce a custom HCL function** (or functions) to represent shell execution and possibly other runtime dynamics. For instance, we can define an HCL function `sh("cmd")` that will serve as a placeholder for "run this command at task runtime." In the HCL **evaluation context**, we'll register `sh()` such that it doesn't actually spawn a process when decoding. Instead, the function can return a special marker value or simply

an empty string, and record the command. One approach: have `sh("echo hi")` return an empty string or some identifiable token via the HCL eval engine, and simultaneously our function implementation can capture the string `"echo hi"` into a map of pending shell commands keyed by variable name. This is a bit tricky because inside an HCL expression we might not easily know the variable name being assigned. Another approach: require the HCL to use a specific shape for dynamic vars (like the YAML-like object form). For example, allow:

```
vars = { VERSION = { sh = "git rev-parse HEAD" } }
```

If our `Vars` decoding is flexible enough, this could populate Var.Sh. We'd need to tell the HCL decoder that the value of `vars` map can be either a string or an object with an `sh` field. HashiCorp's `hcl.Decode` might support this if `Var` struct has fields and possibly implements a custom decode. In fact, Terraform's HCL is capable of decoding a union of types (via custom logic). **Initially, we might opt for a simpler route:** treat anything that isn't a plain literal as something we handle later. We could enforce that in HCL, dynamic shell values must be set via the `sh("...")` function call to avoid ambiguity. For example:

```
task "build" {
  vars = {
    GIT_COMMIT = sh("git log -n1 --format=%H")
  }
}
```

Using a function clearly differentiates it from a static string. We then implement `sh()` in the HCL eval context to store `"git log -n1 ..."` as the command for `GIT_COMMIT` variable's `Sh` field, and return a dummy value (like `""`) for the HCL decode (so that the HCL parser won't error out expecting a string vs object). Later, when the task runs, Task sees that `GIT_COMMIT` Var has `Sh` set and executes it (just as it would in YAML mode) [26] . This ensures the command runs at the proper time. In essence, **we integrate HCL's function mechanism with Task's lazy var resolution**. HCL's design explicitly allows "with support from the calling application, use of variables and functions for more dynamic configuration" [3] . This is exactly our use-case – Task will be the "calling application" that provides a custom function.

Concretely, we will create an `EvalContext` for hclsimple with a `Functions` map containing our `sh()` (and possibly others like `env()` for environment lookup). The `sh` function can be implemented in Go to satisfy HCL's function signature (likely returning an HCL cty Value). For now, we can make it return an empty string (cty.StringVal("")) just to satisfy type, and log the command string along with some identifier of which variable is being set. If identifying the variable name is complicated in the function (since HCL just passes the argument), another approach: define a **dummy type** for shell commands – e.g., have `Var.Static` be empty and `Var.Sh` carry the command. We could hack the function to return a specially crafted string like `"__TASK_SH__:<command>"` and after decode, scan through Vars for values with that pattern to populate the `Sh` field. While not elegant, it's effective and isolates complexity to the parsing stage. A cleaner (but more involved) solution is implementing a **custom HCL decode logic for the Vars map**: we could implement the `hcl.Decoder` interface on our `Vars` type to manually parse the HCL AST for that block, detect `sh()` calls, etc. For an initial implementation, we can go with the simpler function placeholder approach, given that **"for now, simple functionality is fine; more can be added later"** (the user explicitly said we only need basic syntax support first). So initially, we might even decide to *not*

implement dynamic shell var in HCL at all (requiring users to rely on YAML includes if they need it), or implement just enough for common cases.

Aside from `sh()`, we can also provide other utility functions in HCL if desired, analogous to Sprig functions. For example, an `env("VAR")` function to import environment variables (which YAML accomplishes with either `$VAR` usage or the `env` template function). We could map `env("HOME")` to `os.Getenv` at parse time. However, that would capture the environment when parsing, which might be acceptable (since Task's YAML `{{env "HOME"}}` is also evaluated at task runtime, not parse, but environment is usually static per run). To keep things consistent, environment variables can still be passed via the `env` section in Taskfile rather than hard-coded in config, so a function may be optional. Nonetheless, HCL's flexibility allows adding such helpers easily.

In summary, the goal is to **ensure HCL variables/functions are not fully evaluated during decoding** unless they represent static config. HCL will handle pure expressions (like `${NAME}!` concatenation) itself, which is fine, but anything that should happen at runtime (shell execution) we handle as above. Docker Bake's HCL doesn't have the concept of executing arbitrary shell in the config, but Terraform does (via external data sources). Our implementation is simpler – just deferring the execution.

5. Drawing from Terraform & Docker Bake

Since the user mentioned **Terraform and Docker Bake** as references, we'll align our approach with those paradigms: - **Terraform's HCL** uses `variable` blocks for inputs and allows default values and expressions inside config. We will emulate Terraform's style for variables and interpolation. Like Terraform, we'll use HCL's expression powers: for example, one Task variable could be defined in terms of another (`FOO = "${BAR}-baz"` will be resolved by HCL automatically, no manual templating needed). We can leverage HCL's **built-in functions** too (Terraform includes many functions like `upper()`, `join()`, etc.). By default, HCL has a standard library of basic functions (math, string, list ops) that we get for free. We can expose those so that, e.g., `${trimspace(var)}` or `${join(",", list)}` works if needed. This covers a lot of what Task's Sprig functions did. The HashiCorp HCL README demonstrates usage of an `upper()` function provided by the app to uppercase a message [29] – we can provide similar. In fact, we might start with just a small set (maybe mapping to Go's strings functions or using the Terraform functions if available via the `std` package). - **Docker Buildx Bake** uses HCL for defining build targets, and they allow interpolation of variables in strings similarly to Terraform. In Bake, you define `variable "TAG" { default = "latest" }` and use `${TAG}` in a target [30] [31]. We plan the same principle: define Taskfile constants or settings as HCL variables and use them in commands or other fields via `${...}`. This is *far* cleaner than YAML's `{{.VARNAME}}`. HCL interpolation syntax also doesn't suffer from shell-escaping issues the way YAML templates did (e.g., in YAML one had to double quote strings carefully to avoid `{{ }}` braces from being eaten by YAML parser). All that goes away with HCL.

In short, **Terraform's robust variable/expression model will guide our HCL Taskfile design**. We'll use **Terraform-like blocks** for clarity (variable, task, etc.) and rely on **HCL's expression evaluator** to handle references and built-ins. As HashiCorp writes, HCL's model "supports arbitrary expressions" and combines what was previously separate in older versions (HIL for interpolation) into one unified language [32]. We'll be tapping into that power for Taskfile. The result should be a **much more expressive Taskfile** without writing a ton of new logic ourselves.

6. Integrating with VS Code / LSP – `task-hcl-lsp`

To improve the developer experience, we will create a new command (and binary) called `task-hcl-lsp` that implements a basic Language Server Protocol server for HCL Taskfiles. The immediate goal is **simple syntax validation** – when a user opens an `.hcl` Taskfile in their editor, the LSP should parse it and report any syntax or schema errors (with line/column info) so they get feedback in real-time. We won't attempt advanced features initially (no auto-complete or refactoring support yet), focusing on correctness and avoiding false positives.

Why an LSP? Many editors (VSCode, Vim/Neovim, etc.) can use LSP servers for language intelligence. HashiCorp provides an official Terraform LSP (`terraform-ls`) that offers completion/validation for `.tf` files [33]. We'll build a similar tool for Taskfile HCL. This can likely be more lightweight than Terraform's, since Taskfiles are simpler. We also have the advantage of a clear schema (the Taskfile structure) to validate against.

Implementation approach: - The `task-hcl-lsp` will be a Go program (possibly integrated into the Task CLI as a subcommand, or a separate binary in the repo). We can use an existing LSP library (like the Go `x/tools` `jsonrpc2` and `protocol` packages) or a minimal framework. Given "simple syntax validation" is the current requirement, we can even get away with a very minimal implementation: handle `initialize` and `textDocument/didOpen` / `didChange` events by running the HCL parser and capturing errors. - Specifically, on receiving a document (the Taskfile content), we call our HCL decoding logic (or a variant of it that doesn't require a file on disk, since the text may be unsaved – `hcl.Parse()` can handle raw bytes). We will create an HCL *file* parser via the lower-level API (`hclsyntax.ParseConfig()` for example) to get detailed diagnostics. If the parse succeeds and decode to our struct passes, we respond with no errors. If there are syntax errors or schema violations, the HCL library will typically return `hcl.Diagnostics` which include each error's range (start line/col and end) and message. We can convert those to LSP **Diagnostics**. The nice thing is the HCL library is built with good error messages since it's used in Terraform (and it can even suggest "did you mean X?" for some mistakes). We will preserve those and surface them in the editor. - Initially, we do not plan to implement auto-completion of task names or properties. However, we can add basic completion of keywords (like suggest `cmds`, `deps`, etc. within a `task` block) by leveraging the schema knowledge. This could be an enhancement later. The **first priority is validating the file structure and giving errors** (e.g., if a required field is missing or a typo in a field name). HCL's decoder will error on unknown attributes by default, which helps (for instance, if someone writes `comds = [...]` instead of `cmds`, the decoder can flag unknown attribute "comds"). This addresses a common frustration when writing YAML Taskfiles without a schema – here we'll effectively have the schema built into the decoder.

- The LSP will likely need to handle standard requests: `initialize`, `initialized`, `textDocument/didOpen`, `textDocument/didChange`, and `textDocument/close`. For each change or open, we re-parse the document (which is fast for relatively small Taskfiles). We send back diagnostics (errors) if any. We also handle `shutdown` / `exit` messages properly. Since this is a custom LSP, we'll document how to configure editors to use it (e.g., VSCode can be pointed to the `task-hcl-lsp` binary via an extension or `settings.json`).

- In terms of editor integration, we might initially provide instructions for VSCode users to add our LSP manually. In the long run, we could contribute to the existing **Task VSCode extension** (if one exists) or publish a new one for HCL Taskfiles. Notably, Task's docs mention an official VSCode extension that provides YAML Taskfile schema validation via a JSON schema [34] [35]. For HCL, a JSON schema

approach is not applicable, so an LSP is the correct path (Terraform also uses an LSP instead of JSON schema). We can start with the standalone LSP binary approach (it's easier to iterate on).

Summary of LSP "basic functionality": The user will get real-time syntax checking. For example, if they forget a closing brace or use an unknown key, a red squiggly will appear with an error message from our parser. This addresses the user's desire for a "proper LSP" to accompany the new syntax (likely to make adoption smoother by providing IDE support). In the future, we can enhance this LSP with features like autocompletion of task names (useful when writing `deps = ["..."]` to list available tasks), hover tooltips (e.g., show the task description on hover of a task name), or go-to-definition for tasks, etc. Initially, though, we'll keep it *simple and correct* [33].

7. Testing and Compatibility

We will thoroughly test the HCL integration to ensure parity with YAML. Key areas: - **Parse Tests:** We'll create equivalent Taskfiles in YAML and HCL and load both, then compare the resulting `Taskfile` structs. They should be identical. For instance, a YAML with a static var and an HCL with the same var should produce the same `Vars` map internally. We'll test static vars, dynamic vars (if implemented), various data types (strings, bools, ints, lists in YAML vs their HCL representation), and edge cases (empty lists, multiline commands, etc.). We also test error cases – e.g., an HCL Taskfile missing `version` should error (since version is required), or a typo in a field name should be caught by HCL's strict decoding. - **Execution Tests:** Using the same test suite that exists for YAML Taskfiles to run tasks, but feeding it HCL Taskfiles. Because the engine is reused, most tasks should behave identically. We will pay special attention to ordering and lazy evaluation: e.g., a task with `vars: { TIMESTAMP = {sh: "date"} }` in HCL should run that command only when the task runs and not before. If possible, we simulate scenarios (maybe by having the shell command create a file and verifying its timestamp relative to parse time). - **Include Tests:** Try includes mixing formats. For example, an HCL main including a YAML included Taskfile – verify the tasks from the included file appear and run. And the reverse: YAML including HCL. Also test passing variables in includes between formats (the Include struct's `vars` is a map of strings in YAML – if the main file is HCL, ensure we properly decode the HCL `include` block's vars and then apply them when merging includes). The merging logic prohibits includes inside included files [14] – we ensure this still holds with HCL (likely by having the HCL decoder also error if an included file had its own includes). - **LSP Manual Testing:** Run the `task-hcl-lsp` against some sample files to see that it starts and returns diagnostics correctly. We can use `vim` or VSCode with a custom config to connect to it and verify it flags errors. This will likely involve some iteration on offset calculations (to match LSP's UTF-16 vs HCL's rune positions, etc., but that's implementation detail).

8. Impact on Non-HCL (YAML) Users

Our changes will be additive. If a user continues to use YAML Taskfiles, nothing changes. We must ensure that adding HCL support doesn't inadvertently break YAML parsing. Because we might add `hcl` tags to structs and import the HCL library, we should be cautious that YAML unmarshalling still works as before (it should, since yaml library will ignore unknown tags and focus on `yaml:` tags). We will keep all existing YAML tests green. The binary size will grow slightly due to the HCL dependency, but that's acceptable considering the feature gains.

If for some reason the HCL integration required a drastic refactor of core logic (which we are *avoiding*), we would reassess the value. But as outlined, most core code stays the same. We essentially plug in a new parser front-end. This isolated approach means maintenance overhead is low – whenever new features are

added to Taskfile (e.g., new fields or experiments), we just need to add the corresponding HCL tag or handling to support them in the HCL parser.

9. Future Enhancements

Once the basic HCL support is in place and a rudimentary LSP is working, we can enhance both: - **Advanced LSP features:** auto-completion of available tasks, completion of variable names or functions, documentation on hover (for example, hover over `ignore_error` could show a tooltip from Task docs). We could use the schema reference [8] to supply descriptions. This would involve maintaining a schema model in the LSP. We can also integrate with the VSCode extension (perhaps the extension can detect `.hcl` and route to our LSP, or we merge capabilities). - **Refine dynamic evaluation:** We might implement more elegant lazy eval for `sh()` or even allow loops/conditionals if needed (HCL has constructs like `for` expressions and `if` expressions which we could leverage for generating lists of tasks or conditional definitions – though we should be careful not to overcomplicate Taskfile logic). - **Terraform Integration:** The user suggested Terraform as a model; one idea could be to allow importing Terraform-style variables or reuse HCL tooling. For instance, Terraform's language server might be adaptable, but given Taskfile's schema is different, writing our own is simpler.

By modeling our implementation on Terraform and Docker Bake, we ensure we aren't reinventing the wheel. **Docker Bake already demonstrated multi-format support** – it accepts HCL, YAML, JSON for the same underlying data structures [36] [37]. We're doing exactly that for Task: the **same tasks and workflow, just defined in HCL or YAML as the user prefers**.

Conclusion

In summary, we will **fork go-task to add HCL Taskfile support** with minimal disruption to the existing code. HCL will bring robust native templating (variables, functions, conditionals) to Taskfile, eliminating the need for most of the current string templating hacks. We outlined how to parse HCL using HashiCorp's libraries to populate Task's internal structs (leveraging `hclsimple.Decode` for direct struct decoding [20]). YAML and HCL Taskfiles will coexist and even intermix (via includes). We will also deliver a basic `task-hcl-lsp` tool so that users get immediate feedback on HCL syntax errors in their editors – improving the authoring experience significantly. This LSP will initially handle validation (with the potential to expand capabilities later, akin to Terraform's language server [33]).

By keeping the core engine the same and focusing on translating HCL into the existing Taskfile schema, we ensure that if this integration becomes too cumbersome (i.e. requiring pervasive changes), we can evaluate its worth. However, given HCL's design for exactly this purpose (creating config languages), the integration should be straightforward. HCL is expressly built to be embedded into CLI tools and offers both human-friendly syntax and machine parseability [38] [39]. It will let us write Taskfiles that are easier to maintain and more powerful (with built-in expression support). This addresses the user's frustrations and modernizes Task's configuration format without sacrificing the simplicity of the Task tool.

Finally, all these changes will be done in a fork for now, which can be used as a proof-of-concept. If successful, we could propose it upstream to the go-task project. The addition of HCL could be a compelling improvement for the community, offering a choice between the familiar YAML and the more expressive HCL, much like how Docker allows either but pushes advanced users toward HCL for complex scenarios [4].

This plan provides a clear path to implement and iterate on HCL support, and we'll produce an **actionable code roadmap** (with the points above) for engineers or AI coding assistants (like Claude or Codex) to follow in implementing the changes.

Sources:

- Taskfile YAML limitations discussed by users [1].
- HashiCorp's HCL design and advantages over YAML [2] [3].
- Docker Buildx Bake recommending HCL for advanced config [4] and example of variables in HCL [30] [31].
- HCL parsing and decoding via Go libraries [20] [39].
- Taskfile schema reference (YAML) for mapping fields to HCL [8] [15].
- Terraform's language server as inspiration for LSP [33].
- HCL expression examples (variables, interpolation, functions) [5] which we'll leverage in Taskfile.

---

[1] Task is turning yaml into a programming language · go-task task · Discussion #801 · GitHub
https://github.com/go-task/task/discussions/801

[2] [3] [5] [22] [23] [24] [25] [29] [32] [38] [39] GitHub - hashicorp/hcl: HCL is the HashiCorp configuration language.
https://github.com/hashicorp/hcl

[4] [10] [30] [31] Bake file reference | Docker Docs
https://docs.docker.com/build/bake/reference/

[6] [7] [8] [9] [11] [12] [13] [15] [16] [17] [18] [26] [27] Schema Reference | Task
https://taskfile.dev/reference/schema/

[14] read package - github.com/go-task/task/internal/taskfile/read - Go Packages
https://pkg.go.dev/github.com/go-task/task@v2.2.0+incompatible/internal/taskfile/read

[19] Usage | Task
https://taskfile.dev/usage/

[20] hclsimple package - github.com/hashicorp/hcl/v2/hclsimple - Go Packages
https://pkg.go.dev/github.com/hashicorp/hcl/v2/hclsimple

[21] gohcl package - github.com/hashicorp/hcl2/gohcl - Go Packages
https://pkg.go.dev/github.com/hashicorp/hcl2/gohcl

[28] taskfile package - github.com/go-task/task/internal/taskfile - Go Packages
https://pkg.go.dev/github.com/go-task/task/internal/taskfile

[33] hashicorp/terraform-ls: Terraform Language Server - GitHub
https://github.com/hashicorp/terraform-ls

[34] Support additional metadata in Taskfile YAML schema #1916 - GitHub
https://github.com/go-task/task/issues/1916

[35] Support lowercase(taskfile) & yaml extension (i.e., {Taskfile ... - GitHub
https://github.com/go-task/task/issues/369

36 37 Shipyard | Docker Bake: Storing your Docker build config
https://shipyard.build/blog/docker-bake/